

Classificação de Peças de Roupa (Fashion-MNIST)

com uma Rede Neural Convolutacional

Documentação complementar — Fundamentos de IA

Sumário

1	O dataset Fashion-MNIST	1
1.1	Pré-processamento	2
1.2	Metodologia: treino, validação e teste	2
2	Baseline: SGDClassifier e RandomForestClassifier	2
2.1	Por que um baseline	2
2.2	Preparação dos dados	2
2.3	SGDClassifier	3
2.4	RandomForestClassifier	3
2.5	Metodologia de comparação	3
2.6	Resultados	3
3	Arquitetura da CNN	4
3.1	Convolução (Conv2d)	4
3.2	Ativação ReLU	4
3.3	Max pooling (MaxPool2d)	4
3.4	Flatten e camadas totalmente conectadas (Linear)	5
3.5	Dropout	5
4	Base matemática do treino	5
4.1	Softmax e logits	5
4.2	Função de perda: entropia cruzada	5
4.3	Retropropagação e gradiente descendente	5
4.4	Otimizador Adam	6
5	Glossário de jargões	6
6	Referências	6

Materiais do projeto. O slide de apresentação está em `docs/fashion-mnist-slides.pdf`. Os demais materiais complementares (slides em edição, vídeo e arquivos grandes não versionados no Git) ficam numa pasta do Google Drive, cujo link está em `docs/link-drive.txt`.

1 O dataset Fashion-MNIST

O **Fashion-MNIST** é um conjunto de imagens de **peças de roupa** publicado pela Zalando como um substituto *drop-in* do MNIST clássico: mesmo formato e mesma divisão, porém **mais difícil** e mais representativo de tarefas reais de visão computacional. Contém **70.000 imagens**:

- **60.000** imagens para **treino** e **10.000** para **teste**;
- cada imagem é **28 × 28 pixels**, em **tons de cinza** (1 canal), com valores de intensidade de 0 (preto) a 255 (branco);
- são **10 classes** — Camiseta/top, Calça, Pulôver, Vestido, Casaco, Sandália, Camisa, Tênis, Bolsa e Bota — e o dataset é **perfeitamente balanceado** (6.000 exemplos por classe no treino).

1.1 Pré-processamento

Antes de entrar na rede, cada imagem passa por duas transformações:

1. **ToTensor**: converte a imagem em um tensor de números reais na faixa $[0, 1]$ (divide por 255);
2. **Normalize**: recentraliza os pixels usando a média $\mu = 0\{.\}2860$ e o desvio padrão $\sigma = 0\{.\}3530$ do Fashion-MNIST, aplicando

$$x' = \frac{x - \mu}{\sigma}.$$

Isso deixa os dados com média ≈ 0 e desvio ≈ 1 , o que acelera e estabiliza o treino (os gradientes ficam melhor condicionados).

1.2 Metodologia: treino, validação e teste

Para medir o desempenho de forma honesta, os 60.000 exemplos de treino são divididos em **50.000 para treino** e **10.000 para validação**. A validação é usada para acompanhar o aprendizado e escolher o melhor modelo. O **conjunto de teste é usado uma única vez**, ao final — nunca para ajustar hiperparâmetros. Ignorar essa separação causa **vazamento de dados** (*data leakage*) e infla artificialmente a acurácia reportada.

Aplica-se ainda uma **data augmentation** leve — um recorte aleatório com borda de 2px (**RandomCrop**) e um espelhamento horizontal (**RandomHorizontalFlip**) — **apenas ao conjunto de treino**. Isso cria pequenas variações das peças a cada época e reduz o *overfitting*. Validação e teste são avaliados **sem** augmentation; por isso o conjunto de treino é carregado com dois *transforms* diferentes, reaproveitando os mesmos índices do split para que nenhuma imagem de validação receba augmentation.

Essa mesma divisão (50.000 treino, 10.000 validação, 10.000 teste) é reaproveitada pelos modelos de **baseline** (**SGDClassifier**, **RandomForestClassifier**) descritos na próxima seção — garantindo que os números reportados sejam diretamente comparáveis com os da CNN.

2 Baseline: SGDClassifier e RandomForestClassifier

Além da CNN, o projeto treina dois modelos clássicos de Scikit-Learn — **SGDClassifier** e **RandomForestClassifier** — usados como **baseline**: um patamar de comparação mais simples, que ajuda a mostrar se a complexidade extra da CNN realmente se traduz em ganho de desempenho. Este é também o requisito mínimo de modelos exigido pelo enunciado do trabalho para tarefas de classificação.

2.1 Por que um baseline

Diferente da CNN, esses modelos não enxergam a estrutura espacial da imagem — cada pixel é tratado como um atributo independente, sem noção de vizinhança. Se a CNN não superasse esses baselines mais simples, isso seria um sinal de que a complexidade extra (convolução, mais parâmetros, mais tempo de treino) não estaria trazendo benefício real.

2.2 Preparação dos dados

SGDClassifier e **RandomForestClassifier** esperam uma matriz 2D (**n_amostras**, **n_atributos**), não um tensor de imagem. O pré-processamento é, portanto, mais simples que o da CNN:

1. **Achatamento (flatten)**: cada imagem 28×28 vira um vetor de **784 atributos** (um por pixel).
2. **Escala**: os pixels são usados na faixa $[0, 1]$ (após **ToTensor**, sem a normalização Z-score usada na CNN). Essa escala já é adequada para os dois modelos — o **RandomForestClassifier** é

invariante à escala dos atributos (decide por *splits*, não por distância), e o `SGDClassifier` já trabalha bem com atributos numa faixa pequena e comparável entre si. Por isso, **não se aplica um `StandardScaler` adicional**.

3. **Separação estratificada**: os 60.000 exemplos de treino são divididos em **50.000 treino / 10.000 validação**, preservando a proporção das 10 classes em cada subconjunto (`stratify=y`) — mais rigoroso que uma divisão puramente aleatória.

2.3 SGDClassifier

Um classificador **linear**, treinado por **gradiente descendente estocástico**: a cada passo, ajusta os pesos usando um pequeno lote de exemplos, na direção que reduz o erro. É rápido mesmo em datasets grandes, mas só consegue separar as classes por combinações **lineares** dos 784 pixels — não captura relações não lineares nem a proximidade espacial entre eles.

2.4 RandomForestClassifier

Uma **floresta aleatória**: um conjunto (*ensemble*) de várias árvores de decisão, cada uma treinada em um subconjunto aleatório dos dados e dos atributos. A previsão final é a **votação da maioria** das árvores. Cada árvore consegue aprender combinações simples e não lineares entre pixels (ex.: “se o pixel x é claro e o pixel y é escuro...”), mas ainda sem noção de vizinhança espacial — algo que só a convolução da CNN explora de fato.

2.5 Metodologia de comparação

A mesma divisão treino/validação/teste descrita na seção anterior (“Metodologia: treino, validação e teste”) é reaproveitada aqui: a **validação** escolhe entre os dois baselines (nunca o teste), e o vencedor é avaliado **uma única vez** no conjunto de teste, lado a lado com a CNN.

2.6 Resultados

Modelo	Acurácia (validação)	Acurácia (teste)	F1 macro (teste)
SGDClassifier	0,8452	—	—
RandomForestClassifier (<i>vencedor</i>)	0,8836	0,8731	0,8715
CNN (02_cnn.ipynb)	0,9062	0,9024	0,9011
CNN + tuning (03_tuning.ipynb)	0,9246	0,9204	0,9203

Números da CNN + tuning extraídos de `results/grid_results.json` (grid executado na GPU em 2026-07-22); os demais, dos respectivos notebooks.

O `SGDClassifier` não tem acurácia de teste: como perdeu para o `RandomForestClassifier` na validação, ele nunca toca o conjunto de teste — tocar o teste com um modelo que não foi escolhido violaria a regra de “o teste é usado uma única vez”, mesmo sem ajustar hiperparâmetros a partir dele.

O `RandomForestClassifier` venceu o baseline (88,36% na validação) e, avaliado uma única vez no teste, atingiu **87,31%** de acurácia. A CNN supera esse baseline em cerca de **2,9 pontos percentuais** (90,24% vs. 87,31% no teste) — evidência de que a convolução realmente ajuda a capturar padrões que os 784 pixels tratados independentemente não capturam. O ajuste de hiperparâmetros (`lr = 0,002`, `dropout = 0,2`, `weight_decay = 0`) leva a CNN a **92,04%** no teste, um ganho adicional de 1,8 ponto percentual sobre a configuração fixa — consistente com o que se espera de um *fine-tuning*: o maior salto de desempenho vem da escolha da arquitetura (CNN vs. baseline), não do ajuste fino de hiperparâmetros.

A classe mais difícil para a CNN foi **Camisa** (recall $\approx 0\{, \}628$), frequentemente confundida com Camiseta/top e Pulôver — coerente com a análise exploratória, em que essas três classes já apareciam visualmente parecidas nas imagens médias por classe. Esse é o tipo de padrão que a **matriz de confusão** (ver glossário) evidencia.

3 Arquitetura da CNN

Uma **Rede Neural Convolutacional** (CNN) processa a imagem preservando sua estrutura espacial. Em vez de conectar todos os pixels a todos os neurônios, ela aplica **filtros** pequenos que deslizam pela imagem detectando padrões locais (bordas, curvas), e vai combinando esses padrões em representações mais abstratas.

A arquitetura usada é do tipo **LeNet**, com o seguinte fluxo (formato do tensor canais $\times$ altura $\times$ largura):

#	Operação	Entrada $\rightarrow$ Saída
1	Conv2d(1 $\rightarrow$ 32, 3 $\times$ 3, pad=1) + ReLU	1 $\times$ 28 $\times$ 28 $\rightarrow$ 32 $\times$ 28 $\times$ 28
2	MaxPool2d(2)	32 $\times$ 28 $\times$ 28 $\rightarrow$ 32 $\times$ 14 $\times$ 14
3	Conv2d(32 $\rightarrow$ 64, 3 $\times$ 3, pad=1) + ReLU	32 $\times$ 14 $\times$ 14 $\rightarrow$ 64 $\times$ 14 $\times$ 14
4	MaxPool2d(2)	64 $\times$ 14 $\times$ 14 $\rightarrow$ 64 $\times$ 7 $\times$ 7
5	Flatten	64 $\times$ 7 $\times$ 7 $\rightarrow$ 3136
6	Linear(3136 $\rightarrow$ 128) + ReLU + Dropout(0.5)	3136 $\rightarrow$ 128
7	Linear(128 $\rightarrow$ 10)	128 $\rightarrow$ 10 (logits)

3.1 Convolução (Conv2d)

Cada filtro é uma pequena matriz de pesos k (aqui 3 $\times$ 3) que percorre a imagem. Em cada posição, calcula-se a soma dos produtos entre o filtro e a região correspondente da entrada:

$$y_{i,j} = \sum_m \sum_n x_{i+m, j+n} \cdot k_{m,n} + b.$$

O resultado é um **mapa de características** que realça onde o padrão do filtro aparece. A camada aprende 32 filtros na primeira convolução e 64 na segunda. O **padding=1** acrescenta uma borda de zeros para que a saída mantenha o mesmo tamanho espacial da entrada. Em geral, o tamanho de saída é

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1,$$

onde p é o *padding*, k o tamanho do filtro e s o *stride* (passo).

3.2 Ativação ReLU

Após cada convolução aplica-se a não-linearidade **ReLU** (*Rectified Linear Unit*):

$$\text{ReLU}(x) = \max(0, x).$$

Ela zera valores negativos e mantém os positivos. Sem uma função não-linear, empilhar camadas seria equivalente a uma única transformação linear — a não-linearidade é o que permite à rede aprender relações complexas.

3.3 Max pooling (MaxPool2d)

O *pooling* reduz a resolução espacial pegando o **valor máximo** de cada janela 2 $\times$ 2:

$$y_{i,j} = \max_{0 \leq m, n < 2} x_{2i+m, 2j+n}.$$

Isso diminui o custo computacional, dá uma pequena **invariância a translações** (a peça pode estar levemente deslocada) e resume a informação mais forte de cada região.

3.4 *Flatten* e camadas totalmente conectadas (**Linear**)

Após os blocos convolucionais, o tensor $64 \times 7 \times 7$ é **achatado** em um vetor de 3136 números. Uma camada **Linear** (densa) faz uma combinação linear

$$y = Wx + b,$$

onde W é a matriz de pesos e b o viés. A primeira camada densa reduz para 128 neurônios (com ReLU); a última produz **10 logits**, um por classe.

3.5 *Dropout*

Durante o treino, o **Dropout(0.5)** **desativa aleatoriamente 50%** dos neurônios a cada passo. Isso evita que a rede dependa demais de neurônios específicos, funcionando como **regularização** contra *overfitting*. Na avaliação, o dropout é desligado automaticamente.

4 Base matemática do treino

4.1 *Softmax* e logits

A rede devolve um vetor de 10 **logits** $z = (z_0, \dots, z_9)$. A função **softmax** os transforma em probabilidades que somam 1:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=0}^9 e^{z_j}}.$$

A classe prevista é a de maior probabilidade, $\hat{y} = \arg \max_i \text{softmax}(z)_i$.

4.2 Função de perda: entropia cruzada

Mede o quão distante a distribuição prevista está do rótulo verdadeiro. Para um exemplo cuja classe correta é c :

$$\mathcal{L} = -\log(\text{softmax}(z)_c).$$

Quanto maior a probabilidade atribuída à classe certa, menor a perda. No PyTorch, a **CrossEntropyLoss** combina *softmax* e *log* em uma só operação numericamente estável — por isso a rede emite logits, sem *softmax* explícito.

4.3 Retropropagação e gradiente descendente

O treino ajusta os pesos θ para minimizar a perda. A cada lote:

1. **forward**: calcula as previsões e a perda $\mathcal{L}$;
2. **backward** (*backpropagation*): usa a regra da cadeia para obter o gradiente $\nabla_{\theta} \mathcal{L}$ (a direção de maior aumento da perda);
3. **atualização**: move os pesos no sentido contrário ao gradiente,

$$\theta \leftarrow \theta - \eta \cdot \nabla_{\theta} \mathcal{L},$$

onde η é a **taxa de aprendizado** (*learning rate*, aqui 10^{-3}).

4.4 Otimizador Adam

Em vez do gradiente descendente puro, usamos o **Adam**, que adapta o passo de cada parâmetro combinando estimativas da média (momento de 1ª ordem) e da variância (2ª ordem) dos gradientes recentes. Na prática, converge mais rápido e é menos sensível à escolha da taxa de aprendizado.

5 Glossário de jargões

Época (*epoch*) uma passagem completa por todo o conjunto de treino.

Lote (*batch*) subconjunto de exemplos processados de uma vez antes de atualizar os pesos (aqui, 128 imagens).

Hiperparâmetro valor definido **antes** do treino (taxa de aprendizado, tamanho do lote, nº de filtros), em contraste com os **pesos**, que são aprendidos.

Overfitting quando o modelo decora o treino e generaliza mal para dados novos; combatido com *dropout* e validação.

Regularização técnicas que reduzem o *overfitting* (ex.: *dropout*).

Logit saída bruta da rede, antes do *softmax*.

Gradiente vetor de derivadas parciais da perda em relação aos pesos.

Data leakage (vazamento) usar informação do teste durante o treino/seleção, inflando a acurácia reportada.

Acurácia fração de exemplos classificados corretamente.

Matriz de confusão tabela que mostra, para cada classe real, como o modelo distribuiu suas previsões — útil para ver **quais** classes são confundidas.

AMP (*mixed precision*) uso de números de 16 bits para acelerar o treino em GPU.

Baseline modelo mais simples usado como referência, para avaliar se um modelo mais complexo (aqui, a CNN) realmente compensa a complexidade extra.

Estratificação dividir os dados preservando a proporção de cada classe em cada subconjunto (treino, validação, teste).

Ensemble modelo composto pela combinação de vários modelos mais simples — a floresta aleatória, por exemplo, combina a previsão de várias árvores de decisão.

6 Referências

- H. Xiao, K. Rasul, R. Vollgraf. **Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms**. arXiv:1708.07747, 2017. <https://github.com/zalando-research/fashion-mnist>
- Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. **Gradient-Based Learning Applied to Document Recognition**. Proceedings of the IEEE, 86(11):2278–2324, 1998. (arquitetura LeNet)
- A. Paszke et al. **PyTorch: An Imperative Style, High-Performance Deep Learning Library**. NeurIPS, 2019. <https://pytorch.org>
- F. Pedregosa et al. **Scikit-learn: Machine Learning in Python**. JMLR 12:2825–2830, 2011. <https://scikit-learn.org>
- D. P. Kingma, J. Ba. **Adam: A Method for Stochastic Optimization**. ICLR, 2015.